

Náplň cvičení

- aritmetické operátory
- relační operátory
- větvení

Aritmetické operátory

```
int a=11, b=2, c;
```

```
float x=11, y=2.0, z;
```

```
c=a%b; // c=1
```

```
z=x%y; // CHYBA
```

```
c=a/b; // celočíselné dělení, tzn. c=5
```

```
z=x/y; // z=5.5
```

```
z=x/b; //z=5.5
```

```
z=(float)a/b; // neceločíselné dělení, tzn. z=5.5
```

Operátor	Význam
+	Sčítání
-	Odčítání
*	Násobení
/	Celočíselné i neceločíselné dělení
%	Zbytek po celočíselném dělení

Kombinace operátoru přiřazení s jiným operátorem

Výraz	Ekvivalent
$x \text{ } + = 5;$	$x = x \text{ } + 5;$
$x \text{ } * = x;$	$x = x \text{ } * x;$

Převody z int na float a obráceně

```
int e=25, b;  
float x=6.98, y;  
b = (int)x;  
y = (float)e;  
printf("\nb=%d; y=%f\n\n", b, y);
```

```
b=6; y=25.000000
```

Unární operátory ++ --

Výraz	Ekvivalent
<code>a++;</code>	<code>a=a+1;</code>
<code>++a;</code>	<code>a=a+1;</code>

Unární operátory ++ --

Výraz	Ekvivalent	Hodnoty
b=10;		b=10
a=b++;	a=b; b=b+1;	a=10 b=11
c=a*++b;	b=b+1; c=a*b;	b=12 c=10*12=120
c=a*b++;	c=a*b; b=b+1;	c=10*12=120 b=13
c=a+++b;	c=a+b; a=a+1;	c=10+13=23 a=11

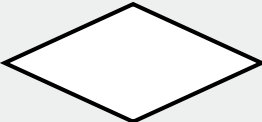
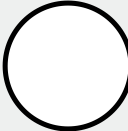
Podmínky – relační operátory

Operátor	Význam	Příklad
<	menší	$a < b$
>	větší	$a > b$
<=	menší nebo rovno	$a \leq b$
<i>nutno dodržovat pořadí obou znaků!!!</i>		
>=	větší nebo rovno	$a \geq b$
<i>nutno dodržovat pořadí obou znaků!!!</i>		
==	rovno	$a == b$
!=	nerovno	$a != b$
<i>je to vlastně negace rovnosti</i>		

Základní komponenty algoritmů

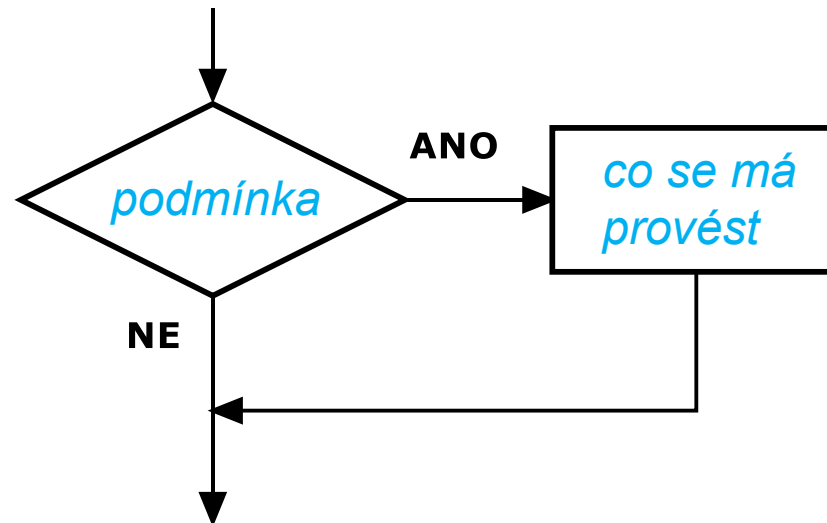
- **Začátek/konec**
- **Posloupnost (sekvence) příkazů**
 - kroky v daném pořadí
 - AKCE
- **Větvení (podmínky)**
 - výběr dalších prováděných kroků závisí na splnění nějaké podmínky
- **Cyklus**
 - opakované provádění kroků
 - a) s pevným počtem opakování
 - b) s podmínkou (na konci/na začátku)

Základní symboly vývojového diagramu

Symbol	Význam
	Začátek/konec algoritmu
	Zpracování, provedení operace
	Vstup/výstup
	Rozhodování
	Cyklus s pevným počtem opakování
	Spojka při rozdělení algoritmu
	Spojovací čára se šipkou – při provádění algoritmu se postupu ve směru šipky

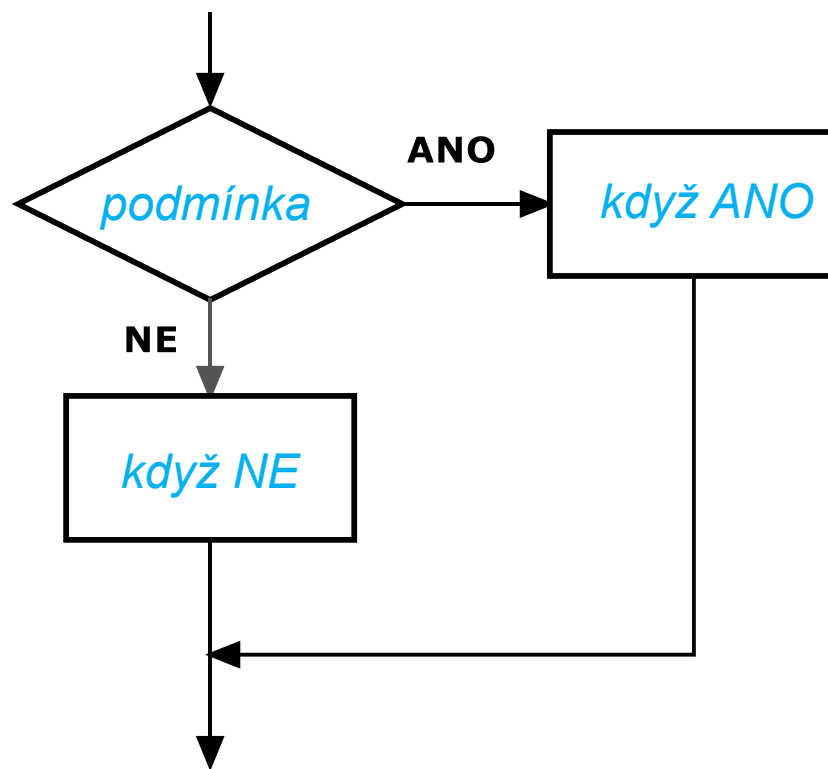
Větvení

- vždy 2 možnosti – podmínka je/není splněna, tzn. z grafického symbolu vycházejí **vždy** 2 šipky



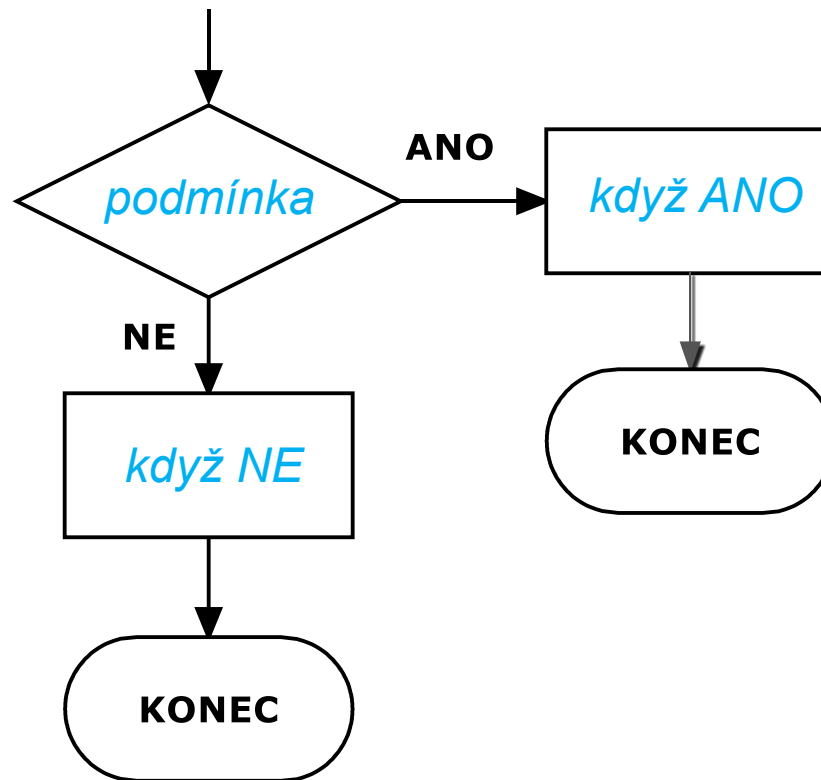
Pokud platí (je splněna) „*podmínka*“,
provede se „*co se má provést*“,
jinak se neprovede nic.

Větvení



Pokud platí (je splněna) „*podmínka*“, provede se „*když ANO*“, jinak se provede „*když NE*“.

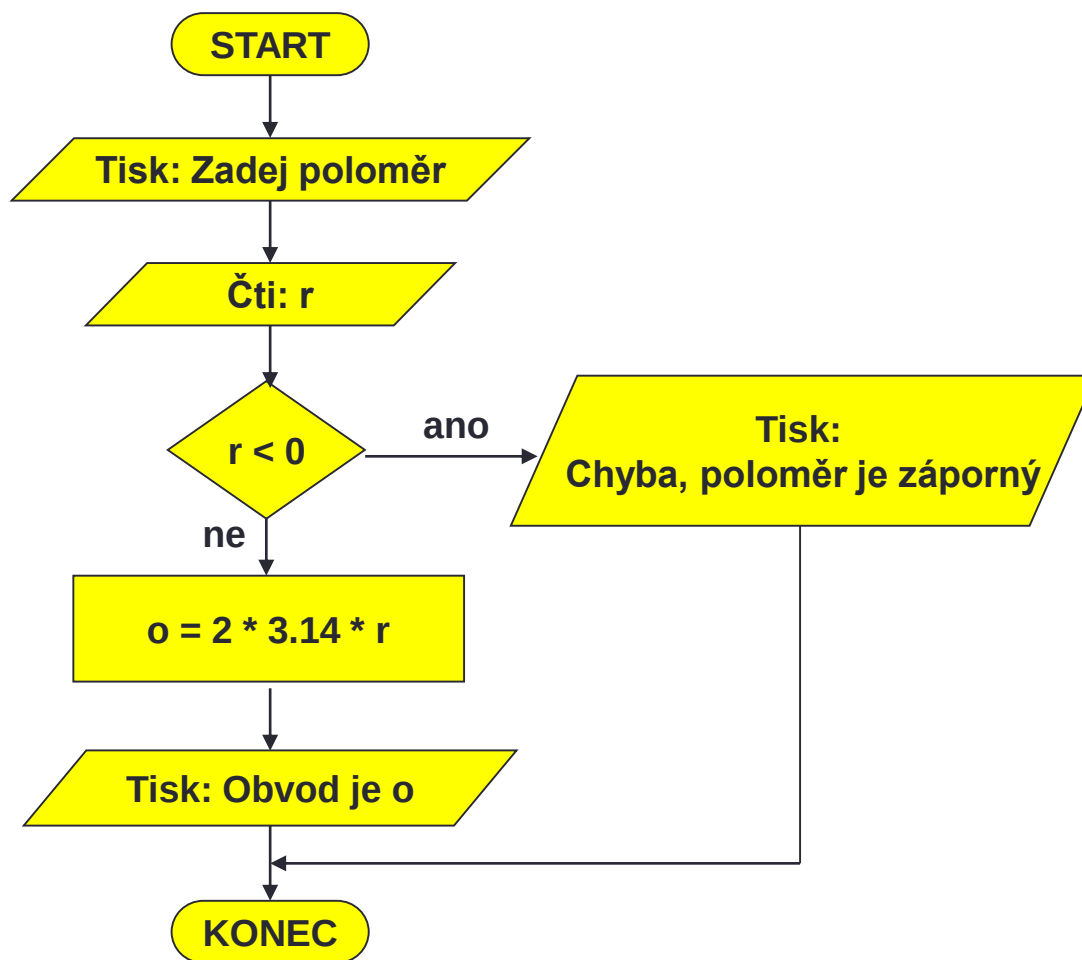
Větvení



Obě větve se nemusí nutně spojit.

Algoritmus pro výpočet obvodu kruhu

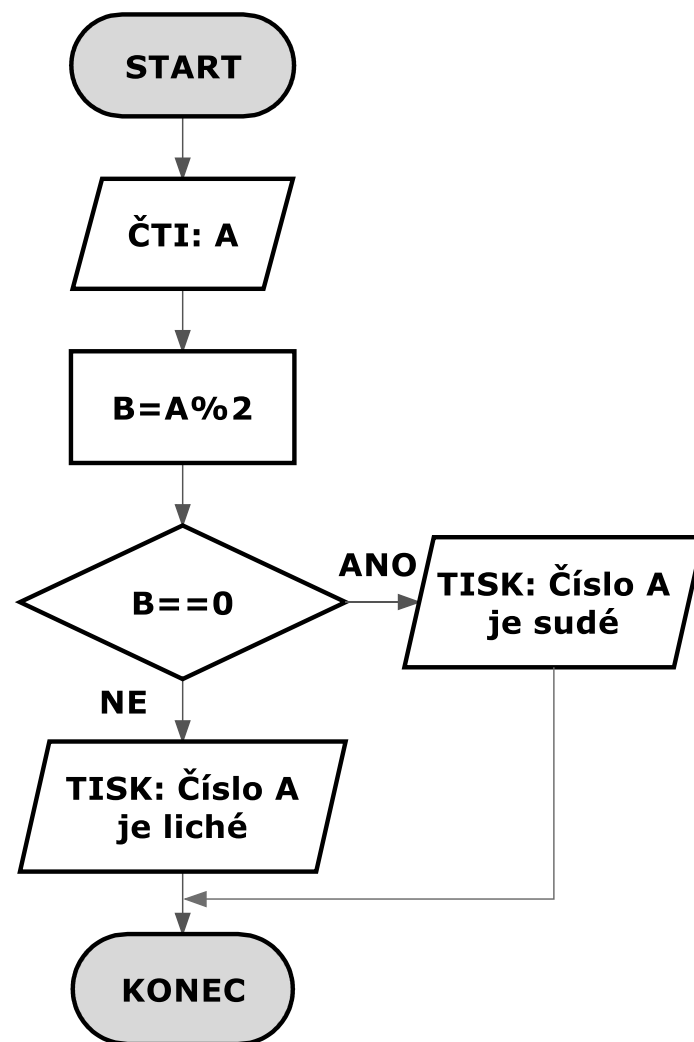
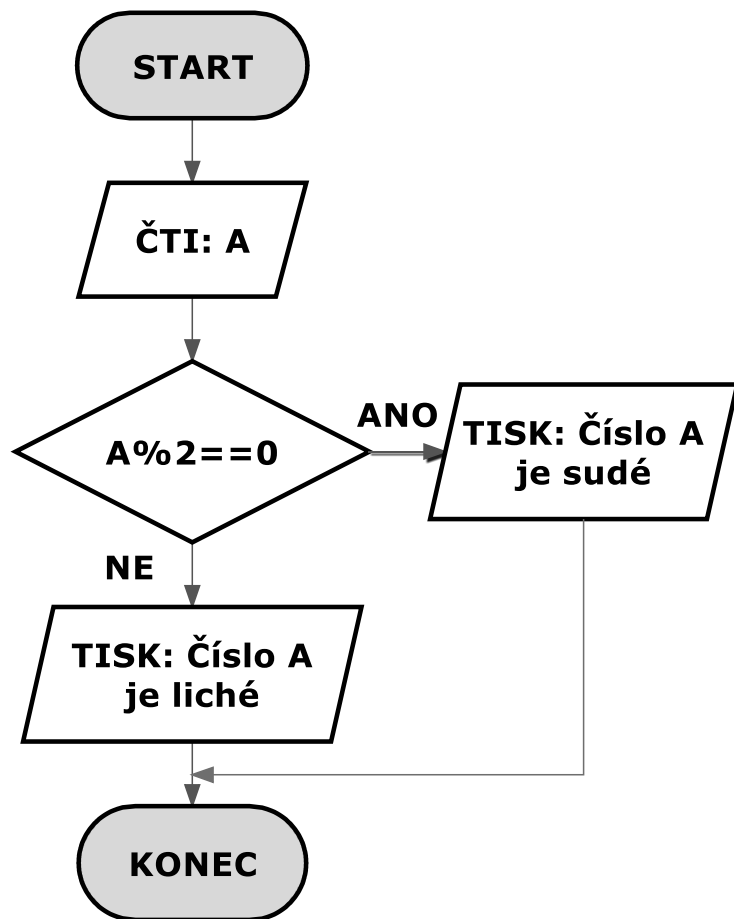
- zápis vývojovým diagramem



Algoritmus pro vyhodnocení, zda zadané celé číslo je sudé nebo liché

- sudá čísla – např. -6, 0, 2, 24, 958
- lichá čísla – např. -5, -1, 1, 25, 1103

Algoritmus pro vyhodnocení, zda zadané celé číslo je sudé nebo liché



Algoritmus pro výpočet čísla zaokrouhleného na desítky

bez použití funkce

Příklady:

vstup

1021

1025

výstup

1020

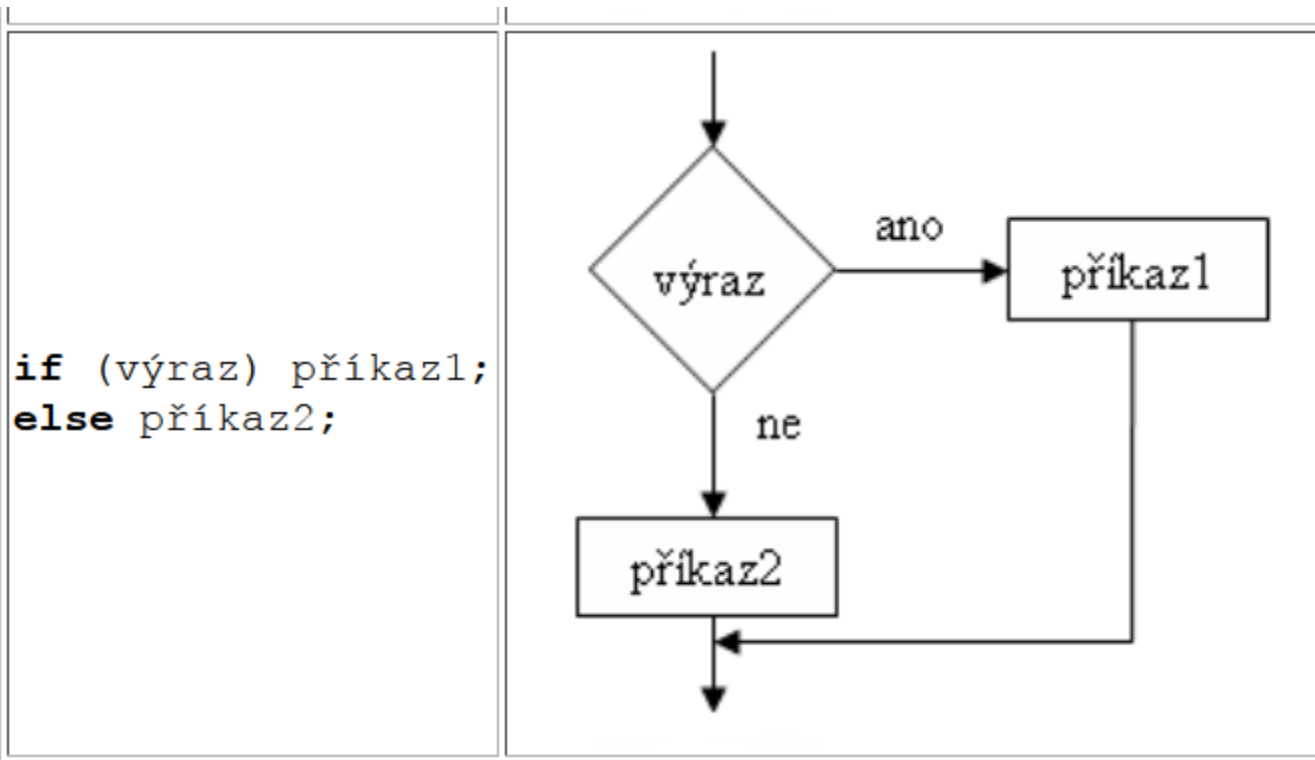
1030

Větvení

Syntaxe	Odpovídající vývojový diagram
<code>if (výraz) příkaz;</code>	<pre>graph TD; Entry(()) --> Výraz{výraz}; Výraz -- ano --> Příkaz[příkaz]; Příkaz --> Join(()); Výraz -- ne --> Join; Join --> Exit(())</pre>

`if (a==b) printf("A se rovna B");`

Větvení



```
if (a==b) printf("A se rovna B");  
else printf("A se nerovna B");
```

Pozor na rovnost!

```
if (r == 0) obvod=0;
```

```
if (r = -1) printf("Nelze vypočítat obvod.");
```

```
/* do proměnné r se vloží hodnota -1,  
podmínka je splněna
```

```
*/
```

Složené podmínky

Spojka	Význam	Příklad
<code>&&</code>	logický součin (AND)	<code>a<b && a<c</code>
<code> </code>	logický součet (OR)	<code>a<b a<c</code>
<code>!</code>	logická negace (NOT)	<code>!(a<b)</code>

- `if (a>b && a>c) ...`
- `if (a>b || a>c && d==0) ...`
- přednost má logický součin před logickým součtem – pro změnu pořadí použít závorky:
- `if (( a>b || a>c) && d==0) ...`

Booleova algebra I.

- logický součet OR, logický součin AND, negace NOT

x	y	x AND y
NEPRAVDA	NEPRAVDA	NEPRAVDA
NEPRAVDA	PRAVDA	NEPRAVDA
PRAVDA	NEPRAVDA	NEPRAVDA
PRAVDA	PRAVDA	PRAVDA

x	y	x OR y
NEPRAVDA	NEPRAVDA	NEPRAVDA
NEPRAVDA	PRAVDA	PRAVDA
PRAVDA	NEPRAVDA	PRAVDA
PRAVDA	PRAVDA	PRAVDA

x	NOT x
NEPRAVDA	PRAVDA
PRAVDA	NEPRAVDA

Booleova algebra II.

- při vyhodnocování složené podmínky má přednost logický součin (AND), pořadí vyhodnocování lze ovlivnit pomocí závorek

Vyhodnoťte následující 2 podmínky:

- P1: $(a > 1) \text{ AND } (b > 1) \text{ OR } (c > 1)$
- P2: $(a > 1) \text{ AND } ((b > 1) \text{ OR } (c > 1))$

a	b	c	P1	P2
4	4	4		
4	2	1		
0	0	2		
4	0	5		
0	0	0		

Booleova algebra III.

Vyhodnoťte následující 2 podmínky:

- P1: $(a > 1) \text{ AND } (b > 1) \text{ OR } (c > 1)$
- P2: $(a > 1) \text{ AND } ((b > 1) \text{ OR } (c > 1))$

a	b	c	P1	P2
4	4	4	PRAVDA	PRAVDA
4	2	1	PRAVDA	PRAVDA
0	0	2	PRAVDA	NEPRAVDA
4	0	5	PRAVDA	PRAVDA
0	0	0	NEPRAVDA	NEPRAVDA

Booleova algebra IV.

- De Morganovy zákony:

$$\text{NOT}(x \text{ AND } y) = \text{NOT } x \text{ OR } \text{NOT } y$$

$$\text{NOT}(x \text{ OR } y) = \text{NOT } x \text{ AND } \text{NOT } y$$

K jakému IF patří ELSE???

if (výraz1)

if (výraz2) příkaz2;

else příkaz3;

if (výraz1)

{ if (výraz2) příkaz2; **}**

else příkaz3

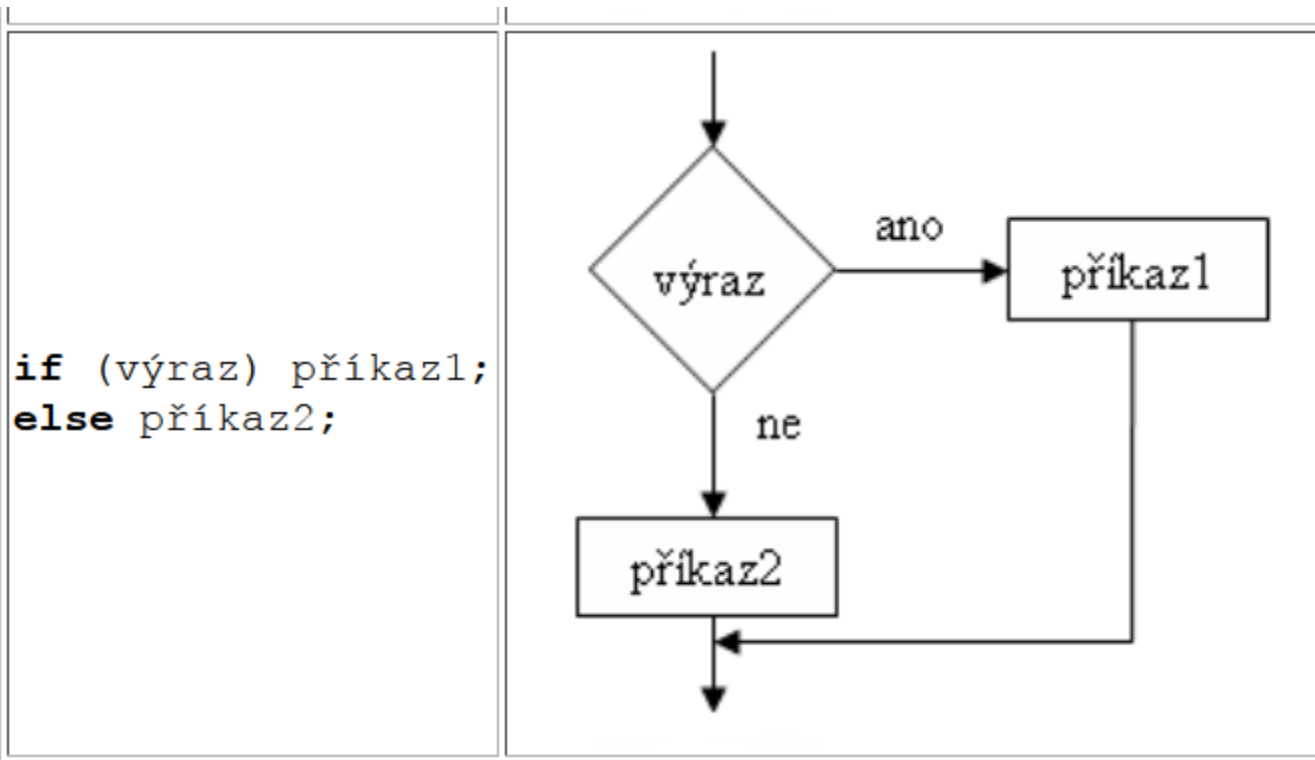
ELSE patří k poslednímu IF

if (výraz1)

if (výraz2) příkaz2;

else příkaz3;

Větvení



```
if (a!=b) printf("A se nerovna B");  
else printf("A se rovna B");
```

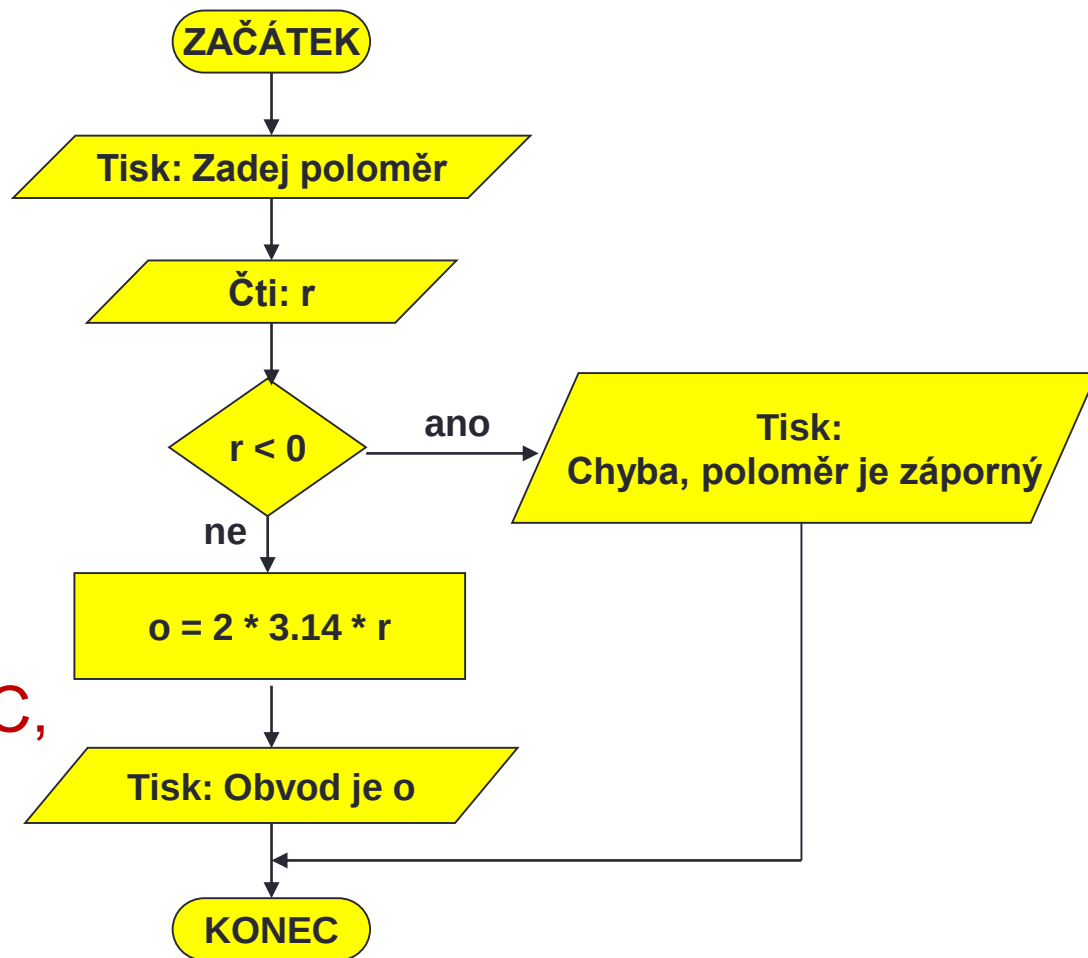
POZOR!!!

```
if (a!=b) printf("A se nerovna B");  
    a=b;      CHYBA  
else printf("A se rovna B");
```

```
if (a!=b) {printf("A se nerovna B");  
        a=b;}  
else printf("A se rovna B");
```

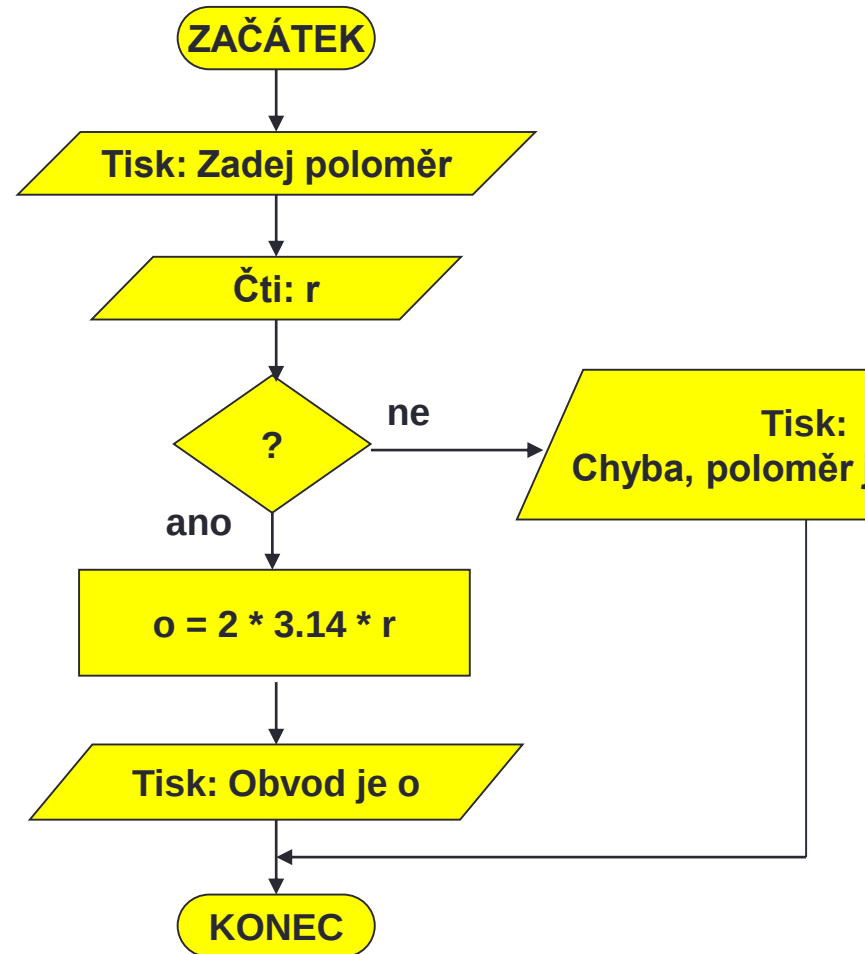
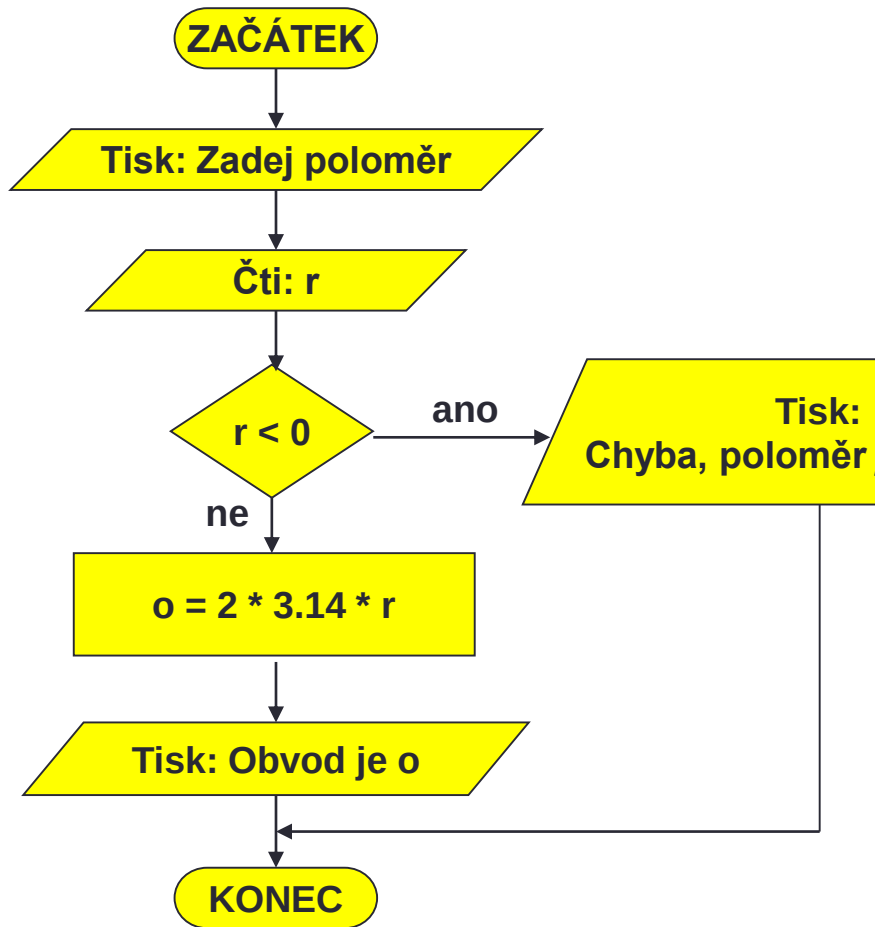
Syntaxe	Odpovídající vývojový diagram
<pre> if (výraz) { příkaz1; příkaz2; } else { příkaz3; příkaz4; } </pre>	<pre> graph TD Start(()) --> Výraz{výraz} Výraz -- ano --> Příkaz1[příkaz1] Příkaz1 --> Příkaz2[příkaz2] Výraz -- ne --> Příkaz3[příkaz3] Příkaz3 --> Příkaz4[příkaz4] Příkaz2 --> Exit(()) Příkaz4 --> Exit </pre>

Algoritmus výpočtu obvodu kruhu

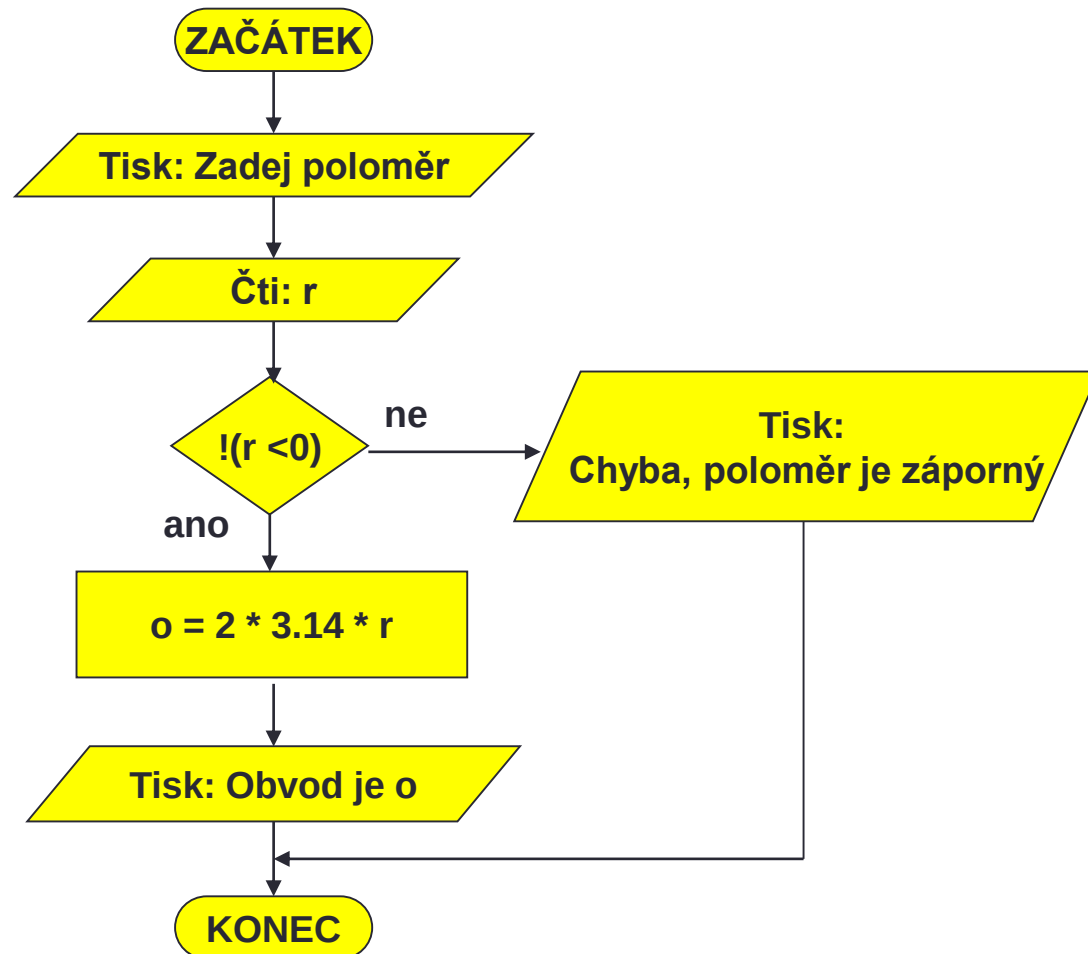


vytvořte program v jazyce C,
proměnná **r** a **o** necht' je
datového typu „double“

Jak upravit podmínku, aby se „ano“ a „ne“ vyměnilo, tzn.



Jak upravit podmínku, aby se „ano“ a „ne“ vyměnilo – řešení negací podmínky

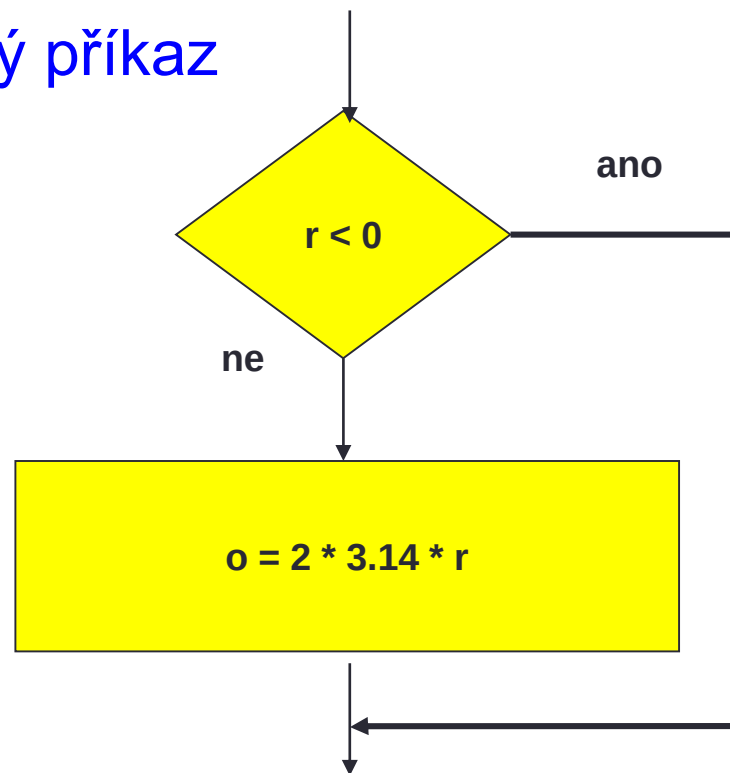


Pozor na středníky – pište jen kam patří!

```
if (r<0) ;  
else { o=2*3.14*r; }
```

prázdný příkaz

**syntaxe v pořádku,
ale důležité je uvědomit
si sémantiku!!!**

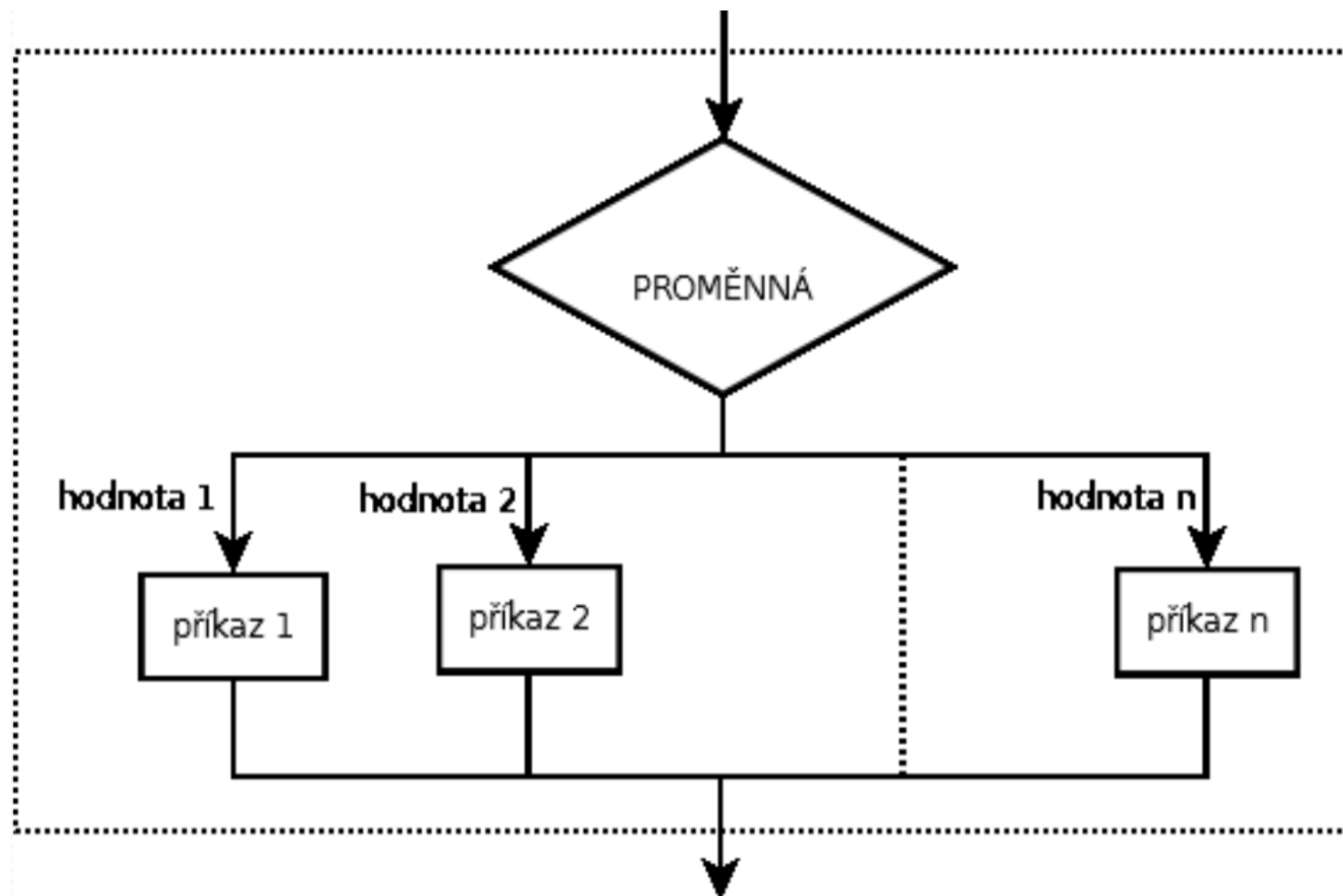


Co vrací funkce scanf

- návratová hodnota je rovna počtu bezchybně načtených a do paměti uložených položek

```
int pocet_polozek, c, d;  
pocet_polozek = scanf("%d-%d", &c, &d);  
if (pocet_polozek==2)  
    printf("Zadano cislo %d a %d.\n", c, d);  
else  
    printf("Nezadal jste 2 polozky, ale pouze: %d.\n", pocet_polozek);
```

SWITCH



SWITCH

```
switch (výraz)          //celočíselný nebo znakový typ
{
    case konstanta1:    //konstantní hodnoty – celé číslo, znak
                        příkaz1;
                        příkaz1a;
                        příkaz1b;
    case konstanta2: příkaz2;
    atd.
    default: příkaz; //default nemusí být
}                      //není středník!!
```

```
switch(znak)
{
    case '1': printf("\nZadal jste 1!");
    case '2': printf("\nZadal jste 2!");
    case '3': printf("\nZadal jste 3!");
}
```

break;

- způsobí „vyskočení“ z těla příkazu switch (a také cyklů, viz příště). Program pak pokračuje ve vykonávání příkazů za blokem switch.

```
switch(znak)
{
    case '1': printf("Zadal jste 1!"); break;
    case '2': printf("Zadal jste 2!"); break;
    case '3': printf("Zadal jste 3!"); break;
    default: printf("Zadal jste něco jiného");
}
```

```
switch(znak)
{
    case 'b': printf("Zadal jste b"); break;
    case 'a':
    case 'A': printf("Zadal jste \"a\" nebo \"A\""); break;
    default: printf("Zadal jste něco jiného");
}
```


return

- uvést KDEKOLIV, kde se má procedura/funkce zcela ukončit
- ve funkci „main“ nezapomeňte za tímto příkazem ještě uvést číslo. „Návratová hodnota funkce main() se vrací operačnímu systému. Zaběhnutá praxe je, že návratová hodnota 0 znamená úspěšné ukončení programu, jakákoliv jiná hodnota určuje číslo chyby (to už je čistě na programátorovi programu, aby vymyslel různým chybám různá čísla).“

Ternární/podmíněný operátor ? :

```
printf("%s\n", (x > y) ? "x je vetsi nez y" : "x je menci ci roven y");
```